

Arrays in Java

Arrays in Java are one of the most fundamental data structures that allow us to store multiple values of the same type in a single variable. They are useful for storing and managing collections of data. Arrays in Java are objects, which makes them work differently from arrays in C/C++ in terms of memory management.

For **primitive arrays**, **elements** are stored in a contiguous memory location,

For **non-primitive arrays**, **references** are stored at contiguous locations, but the actual objects may be at different locations in memory.

Key features of Arrays:

- **Contiguous Memory Allocation (for Primitives):** Java array elements are stored in continuous memory locations, which means that the elements are placed next to each other in memory.
- **Zero-based Indexing:** The first element of the array is at index 0.
- **Fixed Length:** Once an array is created, its size is fixed and cannot be changed.
- **Can Store Primitives & Objects:** Java arrays can hold both primitive types (like int, char, boolean, etc.) and objects (like String, Integer, etc.)

```
public class ArrayDemo
{
    public static void main(String[] args)
    {

        // initializing array
        int[] arr = { 1, 2, 3, 4, 5 };

        // size of array
        int n = arr.length;

        // traversing array
        for (int i = 0; i < n; i++)
            System.out.print(arr[i] + " ");
    }
}
```

Basics of Arrays in Java

There are some basic operations we can start with as mentioned below:

1. Array Declaration

To declare an array in Java, use the following syntax:

type[] arrayName;

- **type:** The data type of the array elements (e.g., int, String).
- **arrayName:** The name of the array.

Note: The array is not yet initialized.

2. Create an Array

To create an array, you need to allocate memory for it using the new keyword:

```
// Creating an array of 5 integers
```

```
int[] numbers = new int[5];
```

This statement initializes the numbers array to hold 5 integers. The default value for each element is 0.

3. Access an Element of an Array

We can access array elements using their index, which starts from 0:

```
// Setting the first element of the array
```

```
numbers[0] = 10;
```

```
// Accessing the first element
```

```
int firstElement = numbers[0];
```

The first line sets the value of the first element to 10. The second line retrieves the value of the first element.

4. Change an Array Element

To change an element, assign a new value to a specific index:

```
// Changing the first element to 20
```

```
numbers[0] = 20;
```

5. Array Length

We can get the length of an array using the length property:

```
// Getting the length of the array
```

```
int length = numbers.length;
```

Now, we have completed with basic operations so let us go through the **in-depth concepts of Java Arrays**, through the diagrams, examples, and explanations.

In-Depth Concepts of Java Array

Following are some important points about Java arrays.

Array Properties

- In Java, all arrays are **dynamically allocated**.
- Arrays may be stored in **contiguous memory** [consecutive memory locations].
- Since arrays are objects in Java, we can find their length using the object property *length*. This is different from C/C++, where we find length using size of.
- A Java array variable can also be declared like other variables with [] after the data type.
- The variables in the array are ordered, and each has an index beginning with 0.
- Java array can also be used as a static field, a local variable, or a method parameter.

An array can contain primitives (int, char, etc.) and object (or non-primitive) references of a class, depending on the definition of the array. In the case of primitive data types, the actual values might be stored in contiguous memory locations (JVM does not guarantee this behavior). In the case of class objects, the actual objects are stored in a heap segment.

40	55	63	17	22	68	89	97	89
0	1	2	3	4	5	6	7	8

<- Array Indices

Array Length = 9

First Index = 0

Last Index = 8

Note: This storage of arrays helps us randomly access the elements of an array [Support Random Access].

Creating, Initializing, and Accessing an Arrays in Java

For understanding the array we need to understand how it actually works. To understand this follow the flow mentioned below:

- Declare
- Initialize
- Access

1. Declaring an Array

The general form of array declaration is

Method

1:

`type var-name[];`

Method

2:

`type[] var-name;`

The element type determines the data type of each element that comprises the array. Like an array of integers, we can also create an array of other primitive data types like char, float, double, etc., or user-defined data types (objects of a class).

Note: It is just how we can create is an array variable, **no actual array exists**. It merely tells the compiler that this variable (int Array) will hold an array of the integer type.

Now, Let us provide memory storage to this created array.

2. Initialization an Array in Java

When an array is declared, only a reference of an array is created. The general form of *new* as it applies to one-dimensional arrays appears as follows:

`var-name = new type [size];`

Here, *type* specifies the type of data being allocated, *size* determines the number of elements in the array, and *var-name* is the name of the array variable that is linked to the array. To use *new* to allocate an array, **you must specify the type and number of elements to allocate**.

Example:

// declaring array

`int intArray[];`

// allocating memory to array

`intArray = new int[20];`

// combining both statements in one

`int[] intArray = new int[20];`

Note: The elements in the array allocated by *new* will automatically be initialized to **zero** (for numeric types), **false** (for boolean), or **null** (for reference types). Do refer to default array values in Java.

Obtaining an array is a two-step process. First, you must declare a variable of the desired array type. Second, you must allocate the memory to hold the array, using new, and assign it to the array variable. Thus, **in Java, all arrays are dynamically allocated.**

Array Literal in Java

In a situation where the size of the array and variables of the array are already known, array literals can be used.

// Declaring array literal

```
int[] intArray = new int[]{ 1,2,3,4,5,6,7,8,9,10 };
```

- The length of this array determines the length of the created array.
- There is no need to write the new int[] part in the latest versions of Java.

3. Accessing Java Array Elements using for Loop

Now, we have created an Array with or without the values stored in it. Access becomes an important part to operate over the values mentioned within the array indexes using the points mentioned below:

- Each element in the array is accessed via its index.
- The index begins with 0 and ends at (total array size)-1.
- All the elements of array can be accessed using Java for Loop.

Let us check the syntax of basic [for loop](#) to traverse an array:

// Accessing the elements of the specified array

```
for (int i = 0; i < arr.length; i++)
```

```
System.out.println("Element at index " + i + " : "+ arr[i]);
```

// Java program to illustrate creating an array

// of integers, puts some values in the array,

// and prints each value to standard output.

```
class ArrayDemo2
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        // declares an Array of integers.
```

```
        int[] arr;
```

```
        // allocating memory for 5 integers.
```

```
        arr = new int[5];
```

```
        // initialize the elements of the array
```

```
        // first to last(fifth) element
```

```
            arr[0] = 10;
```

```
            arr[1] = 20;
```

```
        arr[2] = 30;
```

```
        arr[3] = 40;
```

```
        arr[4] = 50;
```

```
        // accessing the elements of the specified array
```

```
        for (int i = 0; i < arr.length; i++)
```

```
            System.out.println("Element at index "
```

```

        + i + " : " + arr[i]);
    }
}

```

Types of Arrays in Java

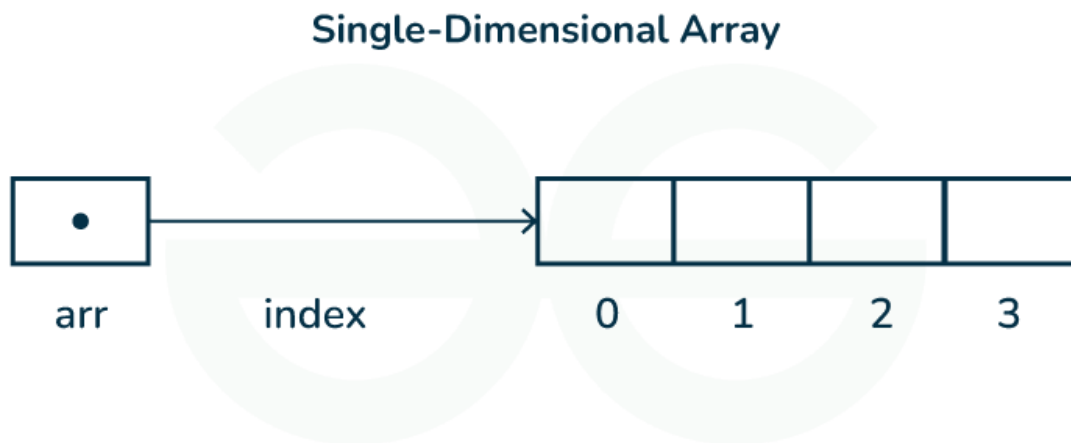
Java supports different types of arrays:

1. Single-Dimensional Arrays

These are the most common type of arrays, where elements are stored in a linear order.

// A single-dimensional array

```
int[] singleDimArray = {1, 2, 3, 4, 5};
```



2. Multi-Dimensional Arrays

Arrays with more than one dimension, such as two-dimensional arrays (matrices).

// A 2D array (matrix)

```
int[][] multiDimArray = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9} };

```

You can also access java arrays using [for each loops](#).

Arrays of Objects in Java

An array of objects is created like an array of primitive-type data items in the following way.

Syntax:

Method 1:

```
ObjectType[] arrName;
```

Method 2:

```
ObjectType arrName[];
```

Example of Arrays of Objects

Example: Here we are taking a student class and creating an array of Student with five Student objects stored in the array. The Student objects have to be instantiated using the constructor of the Student class, and their references should be assigned to the array elements.

```
// Java program to illustrate creating
// an array of objects

```

```

class Student {
    public int roll_no;
    public String name;

    Student(int roll_no, String name){
        this.roll_no = roll_no;
        this.name = name;
    }
}

public class StudentArray
{
    public static void main(String[] args){

        // declares an Array of Student
        Student[] arr;

        // allocating memory for 5 objects of type Student.
        arr = new Student[5];

        // initialize the elements of the array
        arr[0] = new Student(1, "aman");
        arr[1] = new Student(2, "vaibhav");
        arr[2] = new Student(3, "shikar");
        arr[3] = new Student(4, "dharmesh");
        arr[4] = new Student(5, "mohit");

        // accessing the elements of the specified array
        for (int i = 0; i < arr.length; i++)
            System.out.println("Element at " + i + " : { "+ arr[i].roll_no + " " + arr[i].name+" }");
    }
}

```

Multidimensional Arrays in Java

Multidimensional arrays are **arrays of arrays** with each element of the array holding the reference of other arrays. A multidimensional array is created by appending one set of square brackets ([]) per dimension.

Syntax:

There are **2 methods** to declare Java Multidimensional Arrays as mentioned below:

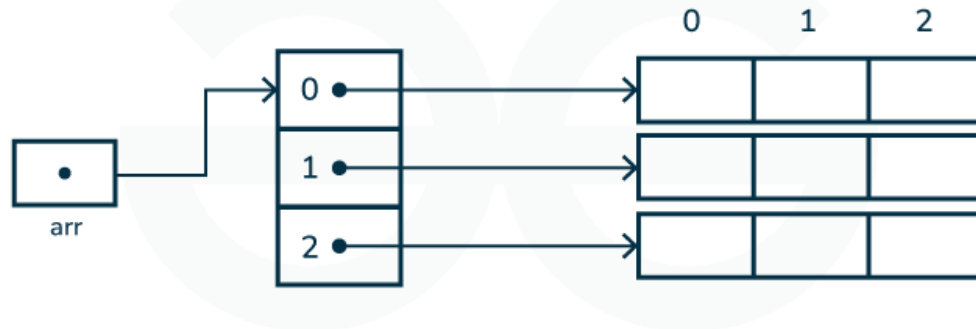
// Method 1

datatype [][] arrayrefvariable;

// Method 2

datatype arrayrefvariable[][];

Multi-Dimensional Array



Declaration:

// 2D array or matrix

```
int[][] intArray = new int[10][20];
```

// 3D array

```
int[][][] intArray = new int[10][20][10];
```

Java Multidimensional Arrays Examples

Example: Let us start with basic two dimensional Array declared and initialized.

// Java Program to demonstrate

// Multidimensional Array

```
import java.io.*;
```

```
class MultiDimensionalArray
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    // Two Dimensional Array
```

```
    // Declared and Initialized
```

```
    int[][] arr = new int[3][3];
```

```
    // Number of Rows
```

```
    System.out.println("Rows : " + arr.length);
```

```
    // Number of Columns
```

```
    System.out.println("Columns : " + arr[0].length);
```

```
}
```

```
}
```

Example: Now, after declaring and initializing the array we will check how to Traverse the Multidimensional Array using for loop.

// Java Program to Multidimensional Array

```
// Driver Class
public class MultiDimensional
{
    // main function
    public static void main(String args[])
    {
        // declaring and initializing 2D array
        int arr[][] = { { 2, 7, 9 }, { 3, 6, 1 }, { 7, 4, 2 } };

        // printing 2D array
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++)
                System.out.print(arr[i][j] + " ");

            System.out.println();
        }
    }
}
```

Passing Arrays to Methods

Like variables, we can also pass arrays to methods. For example, the below program passes the array to method *sum* to calculate the sum of the array's values.

// Java program to demonstrate
// passing of array to method

```
public class ArrayToMethod
{
    // Driver method
    public static void main(String args[])
    {
        int arr[] = { 3, 1, 2, 5, 4 };

        // passing array to method m1
        sum(arr);
    }

    public static void sum(int[] arr)
    {
        // getting sum of array values
        int sum = 0;

        for (int i = 0; i < arr.length; i++)
            sum += arr[i];

        System.out.println("sum of array values : " + sum);
    }
}
```

Returning Arrays from Methods

As usual, a method can also return an array. For example, the below program returns an array from method *m1*.

```
// Java program to demonstrate  
// return of array from method
```

```
class ArrayFromMethod  
{  
    // Driver method  
    public static void main(String args[])  
    {  
        int arr[] = m1();  
  
        for (int i = 0; i < arr.length; i++)  
            System.out.print(arr[i] + " ");  
    }  
  
    public static int[] m1()  
    {  
        // returning array  
        return new int[] { 1, 2, 3 };  
    }  
}
```

Java Array Members

Now, as you know that arrays are objects of a class, and a direct superclass of arrays is a class *Object*.

The members of an array type are all of the following:

- The public final field *length* contains the number of components of the array. Length may be positive or zero.
- All the members are inherited from class *Object*; the only method of *Object* that is not inherited is its *clone* method.
- The public method *clone()* overrides the clone method in class *Object* and throws no checked exceptions.